

Prepared by: Amandeep Gangwar

Course: BCA VI Semester

University: Invertis University

UNIT 1 – Python Database Interaction

1 Mark Questions

Q1. What module in Python is used for connecting to MySQL databases?

Ans: The `mysql.connector` or `pymysql` module is used for connecting Python with MySQL databases.

Q2. What is a database cursor?

Ans: A cursor is an object used to execute SQL queries and fetch results from the database.

Q3. What does the `commit()` function do?

Ans: It saves all the changes made to the database permanently.

Q4. What does the `rollback()` function do?

Ans: It undoes all the uncommitted changes in the current database transaction.

Q5. What is the default port number for MySQL?

Ans: The default port number is **3306**.

Q6. Which Python method is used to execute SQL queries?

Ans: The `execute()` method of the cursor object.

Q7. What is a connection object in Python database programming?

Ans: It represents a connection between the Python program and the database.

🟡 2–3 Mark Questions

Q8. What are the steps for connecting Python with an SQL database?

Ans:

1. Import the database module.
2. Establish a connection using `connect()`.
3. Create a cursor object.
4. Execute SQL queries.
5. Commit changes (if needed).
6. Close the connection.

Q9. Write a Python code to connect to a MySQL database.

```
import mysql.connector
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",
    database="college"
)
if con.is_connected():
    print("Database Connected Successfully")
```

Q10. What is the purpose of the cursor object in database programming?

Ans:

- Executes SQL commands (`execute()`, `executemany()`)
- Fetches query results (`fetchone()`, `fetchall()`)

- Manages database transactions

Q11. How do you create a table in a MySQL database using Python?

```
import mysql.connector
con = mysql.connector.connect(host="localhost", user="root",
password="1234", database="college")
cur = con.cursor()
cur.execute("CREATE TABLE student(id INT PRIMARY KEY, name
VARCHAR(30))")
print("Table created successfully")
```

Q12. What is the difference between `fetchone()` and `fetchall()` methods?

Method	Description
<code>fetchone()</code>	Returns the next single record
<code>fetchall()</code>	Returns all records from result set

Q13. Explain how configuration data can be stored for database access.

Ans:

Config data like host, username, password, and database name can be stored in a `.ini` or `.json` file and loaded dynamically:

```
import json
with open('config.json') as f:
    config = json.load(f)
print(config['user'])
```

5 Mark Questions

Q14. Write a Python program to insert and retrieve data from a MySQL table.

```
import mysql.connector

con = mysql.connector.connect(host="localhost", user="root",
password="1234", database="college")
cur = con.cursor()

# Inserting data
cur.execute("INSERT INTO student VALUES (1, 'Amandeep')")
con.commit()

# Retrieving data
cur.execute("SELECT * FROM student")
for row in cur.fetchall():
    print(row)

con.close()
```

Q15. Explain exception handling in database connections with example.

```
import mysql.connector
try:
    con = mysql.connector.connect(host="localhost", user="root",
password="wrong", database="college")
except mysql.connector.Error as err:
    print("Error:", err)
else:
    print("Connected successfully")
finally:
    if con.is_connected():
        con.close()
```

Q16. Explain how to read and store database config information in Python.

Ans:

We can store database credentials in a JSON or INI file for security.

Example:

config.json

```
{  
    "host": "localhost",  
    "user": "root",  
    "password": "1234",  
    "database": "college"  
}
```

Python code:

```
import mysql.connector, json  
config = json.load(open('config.json'))  
con = mysql.connector.connect(**config)  
print("Connected using config file!")
```

10 Mark Questions

Q17. Explain the full process of database programming using Python with examples.

Ans:

Steps:

1. **Connect** to the database
2. **Create Cursor**
3. **Execute SQL Commands**
4. **Commit or Rollback**
5. **Close connection**

Example:

```
import mysql.connector  
  
con = mysql.connector.connect(host="localhost", user="root",  
password="1234", database="college")  
cur = con.cursor()
```

```
cur.execute("CREATE TABLE IF NOT EXISTS student(id INT PRIMARY KEY,  
name VARCHAR(30))")  
cur.execute("INSERT INTO student VALUES(101, 'Amandeep')")  
cur.execute("SELECT * FROM student")  
  
rows = cur.fetchall()  
for r in rows:  
    print(r)  
  
con.commit()  
con.close()
```

This program demonstrates full database lifecycle management in Python.

UNIT 2 – PYTHON MULTITHREADING

1 Mark Questions

Q1. What is a thread in Python?

Ans: A thread is a lightweight subprocess that allows concurrent execution of tasks.

Q2. Which module is used for multithreading in Python?

Ans: The `threading` module.

Q3. What is the main thread?

Ans: The thread from which all other threads are created when a program starts.

Q4. How do you start a thread in Python?

Ans: By calling the `start()` method of a thread object.

Q5. What is the purpose of `join()` method?

Ans: It ensures that the main thread waits for the child thread to complete execution.

● 2–3 Mark Questions

Q6. Write the syntax to create and start a thread.

```
import threading

def show():
    print("Thread is running")

t = threading.Thread(target=show)
t.start()
```

Q7. What is the difference between process and thread?

Process	Thread
Heavyweight	Lightweight
Has separate memory	Shares memory
Slower	Faster
Created using multiprocessing	Created using threading

Q8. What is synchronization in threads?

Ans: Synchronization ensures that multiple threads do not access shared resources simultaneously to avoid data inconsistency.

● 5 Mark Questions

Q9. Explain thread synchronization with an example.

```
import threading
```

```
lock = threading.Lock()

def display(name):
    lock.acquire()
    for i in range(3):
        print(name, "is running")
    lock.release()

t1 = threading.Thread(target=display, args=("Thread1",))
t2 = threading.Thread(target=display, args=("Thread2",))
t1.start()
t2.start()
```

Explanation:

Here, **Lock** ensures only one thread accesses the code block at a time.

Q10. Explain how to create multiple threads using the **Thread** class.

```
import threading
import time

def worker(id):
    print(f"Worker {id} started")
    time.sleep(2)
    print(f"Worker {id} finished")

threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()
print("All threads completed.")
```

● 10 Mark Questions

Q11. Explain Python Multithreading in detail with examples.

Ans:

Multithreading allows multiple threads to run concurrently, improving performance in I/O-bound programs.

Steps:

1. Import `threading` module
2. Define target function
3. Create thread object
4. Start thread
5. Join threads

Example:

```
import threading, time

def print_numbers():
    for i in range(1, 6):
        print("Number:", i)
        time.sleep(1)

def print_letters():
    for l in ['A', 'B', 'C', 'D', 'E']:
        print("Letter:", l)
        time.sleep(1)

t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

t1.start()
t2.start()
t1.join()
t2.join()
```

```
print("Both threads finished!")
```

Output:

```
Number: 1  
Letter: A  
Number: 2  
Letter: B  
...  
Both threads finished!
```

Conclusion

This set covers **Python Database Interaction** and **Multithreading**, including:

- Connection setup
- CRUD operations
- Exception handling
- Thread creation
- Synchronization